

Digital Engineering

Combinational Circuits

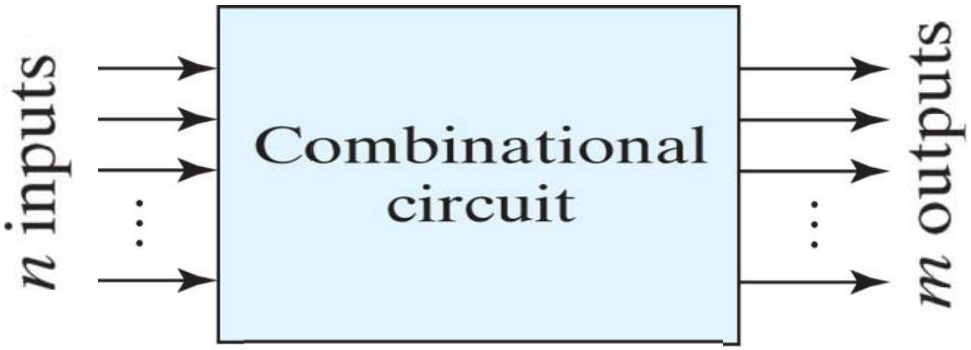
Outlines

- Introduction
- Combinational Circuit Analysis
- Combinational Circuit Design
- Binary Adders and Subtructors
- Binary Decoders and Encoders
- Magnitude Comparator
- Binary Multiplexers
- Three-state Gates

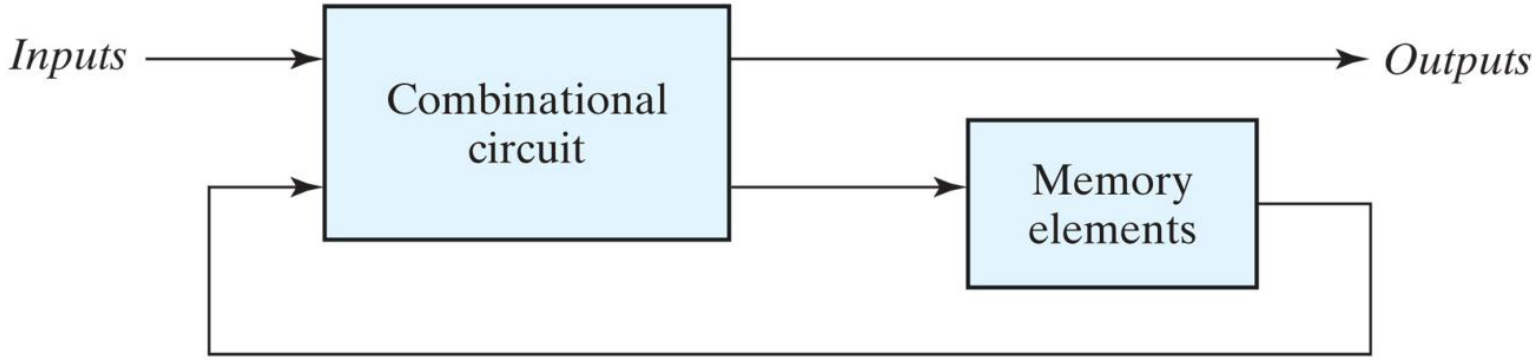
Introduction

Digital Circuits have two types of circuits:

1. Combinational Circuits
   - Outputs depend on the inputs only
   - Memoryless circuits



2. Sequential Circuits
   - Current outputs depend on the inputs and the previous state.
   - Use memory to store the previous state.



Introduction (Cont.)

Combinational Circuits can be:

- Combinational Circuit analysis:
  - In the analysis we are interested in determining the function of a given combination circuit.
- Combinational Circuit design:
  - In the design we are interested in developing a combinational circuit based on a given function.



Combinational Circuit Analysis

Combinational Circuit Analysis

Circuit analysis can be achieved by either one of the following two methods:

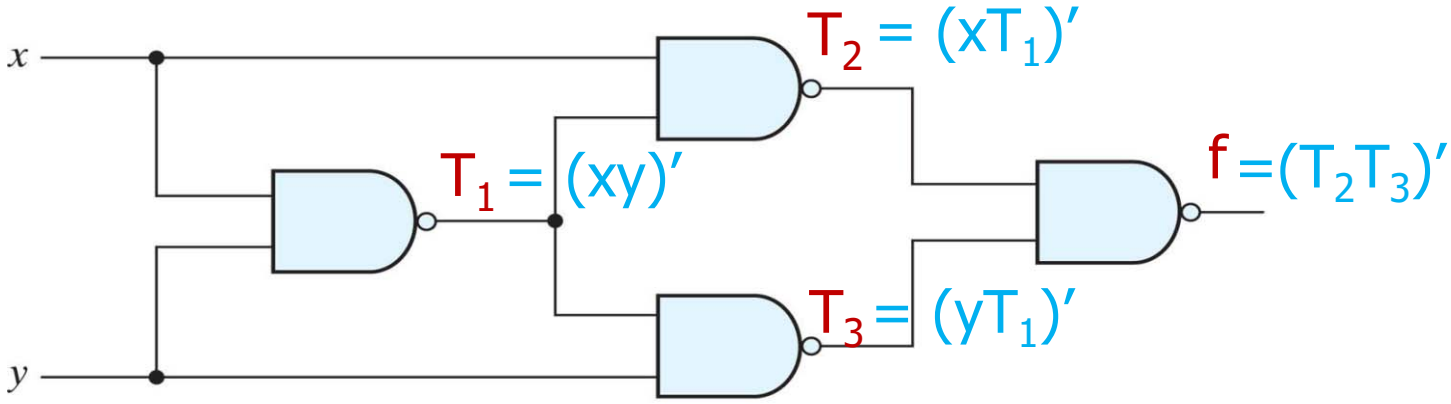
- Determining the output functions as algebraic expressions.

- Determining the truth table of the output functions.

Circuit Analysis steps

1. Label all gate outputs with symbols.
2. Determine Boolean function at the output of each gate.
3. Express functions in terms of input variables

- **Example:** Find the output function of the following circuit.



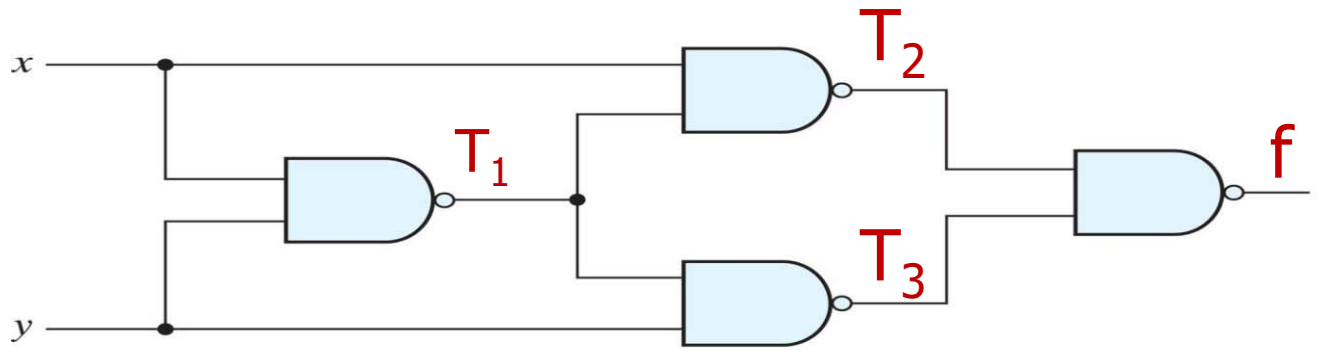
Step$_1$

Step$_2$

Step$_3$ $f = (T_2T_3)' = ((xT_1)'\,(yT_1)')' = (xT_1)+(yT_1) = x(xy)'+y(xy)'$

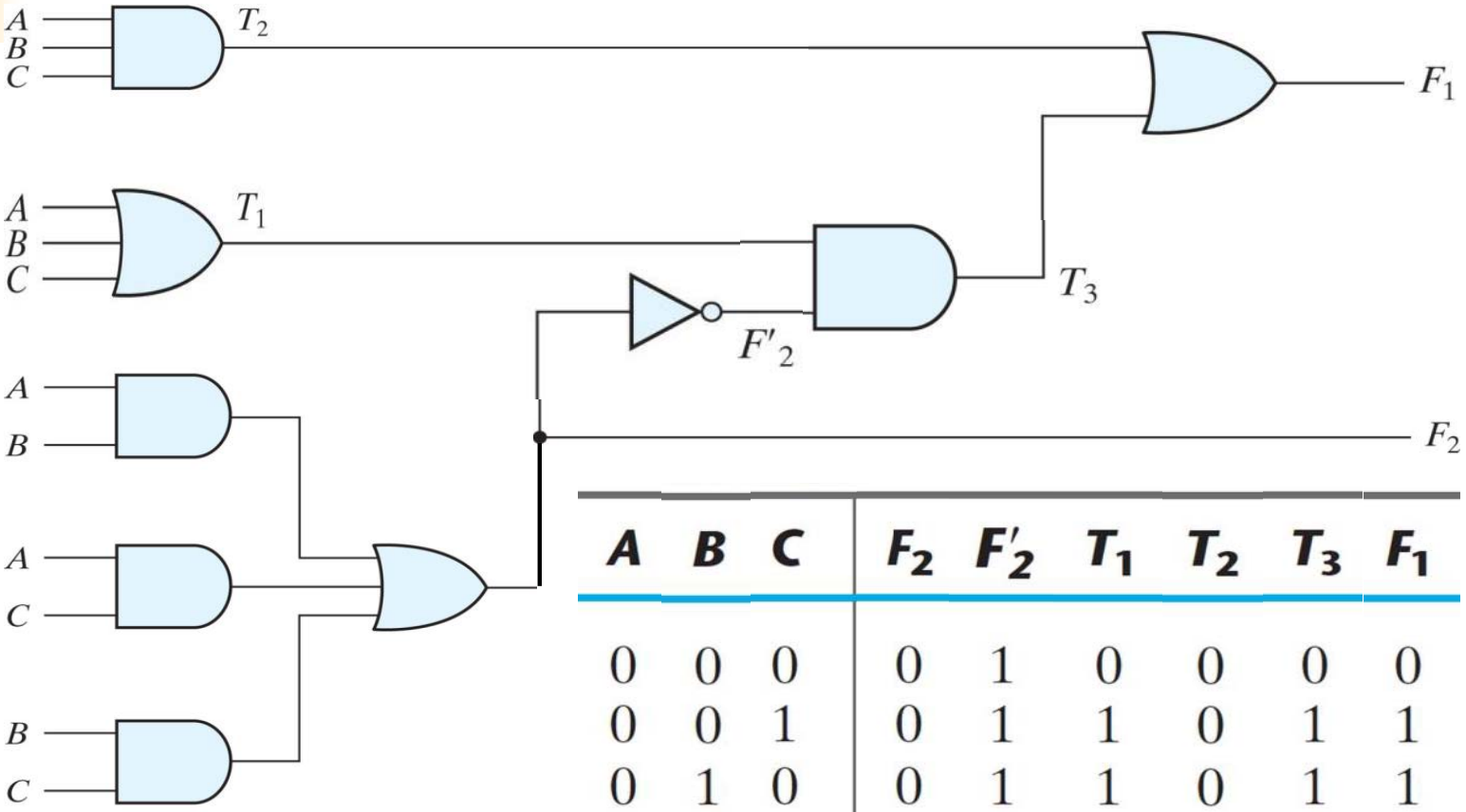
$= x(x'+y')+y(x'+y') = xy' + x'y = x \oplus y$

Solution using Truth Table



x	y	$T_1=(xy)'$	$T_2=(xT_1)'$	$T_3=(yT_1)'$	$f=(T_2T_3)'$
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	0

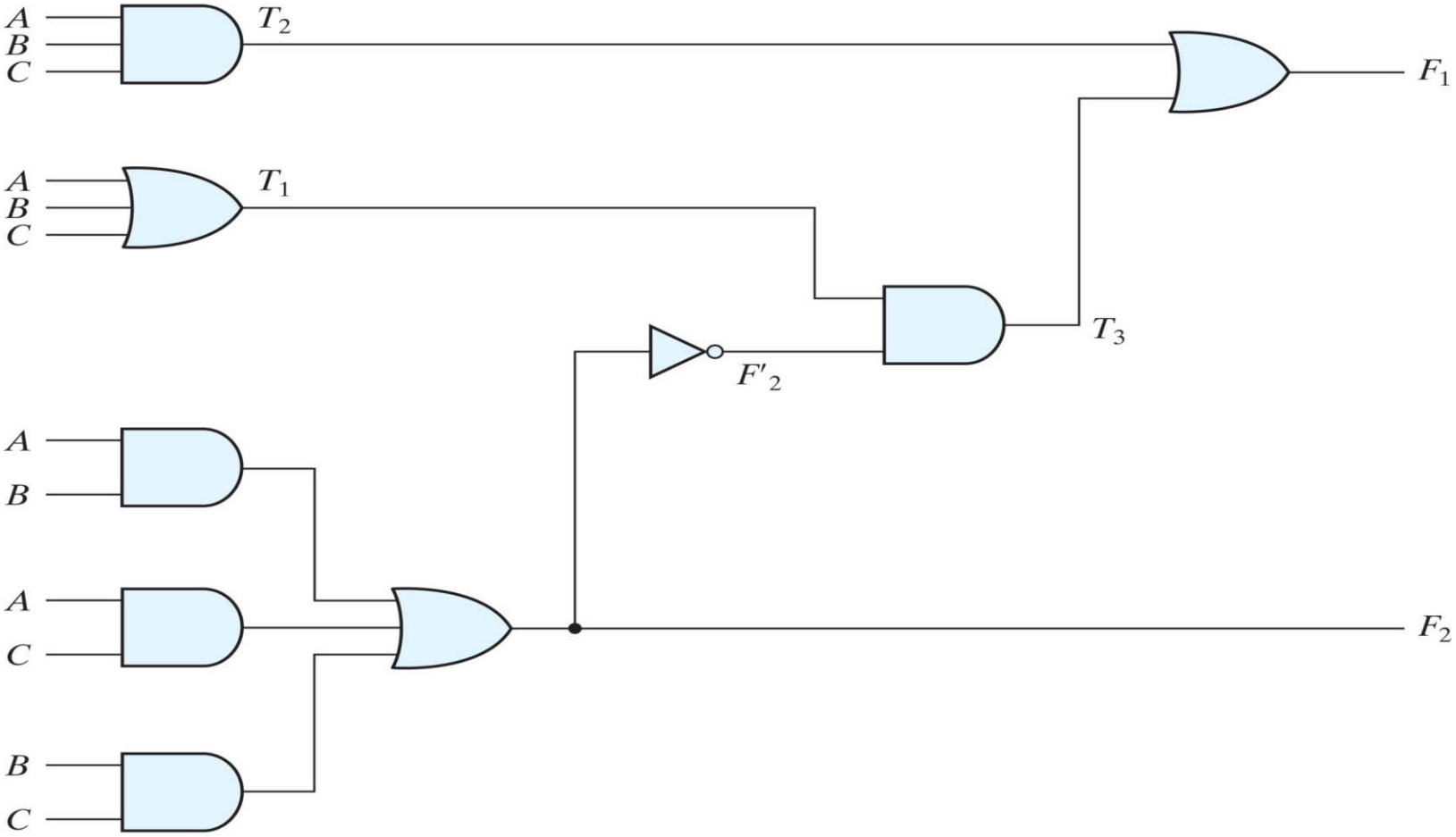
Example 2: Find the functions of the following Circuit using Truth Table



A	B	C	F_2	F'_2	T_1	T_2	T_3	F_1
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

Exercise

What are the output functions F_1 and F_2 of the following logic circuit?



Copyright ©2013 Pearson Education, publishing as Prentice Hall



Combinational Circuit Design

Design of Combinational Circuits

Four main steps in designing a combinational circuit:

1. Determine the number of inputs and outputs and give them symbols.
2. Derive the related Truth Table.
3. Simplify each output.
4. Draw logic diagram and verify correctness.

Design of Combinational Circuits (Cont.)

Example 1: Design a combinational logic circuit that detects odd inputs which are in the rang from 0 to 7.

Solution:

Step 1: 3 inputs (A, B, and C) and one output (F).

Step 2: Truth table

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Step 3: Simplify

A\BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

F = C

Design of Combinational Circuits (Cont.)

Example 2: Develop a combinational circuit that converts a BCD digit into a Excess-3 code.

Solution:

Step 1:

- Inputs: BCD digit
  (4 inputs: A, B, C, D)
- Outputs: Excess-3 code
  (4 outputs: w, x, y, z).

Step 2: Truth table

A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Example 2 (Cont.)

Step 3: Outputs' Simplification

AB\CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	x	x	x	x
10	1	1	x	x

w = A + BC + BD

AB\CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	x	x	x	x
10	0	1	x	x

x = B'C +B'D +BC'D'

AB\CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	x	x	x	x
10	1	0	x	x

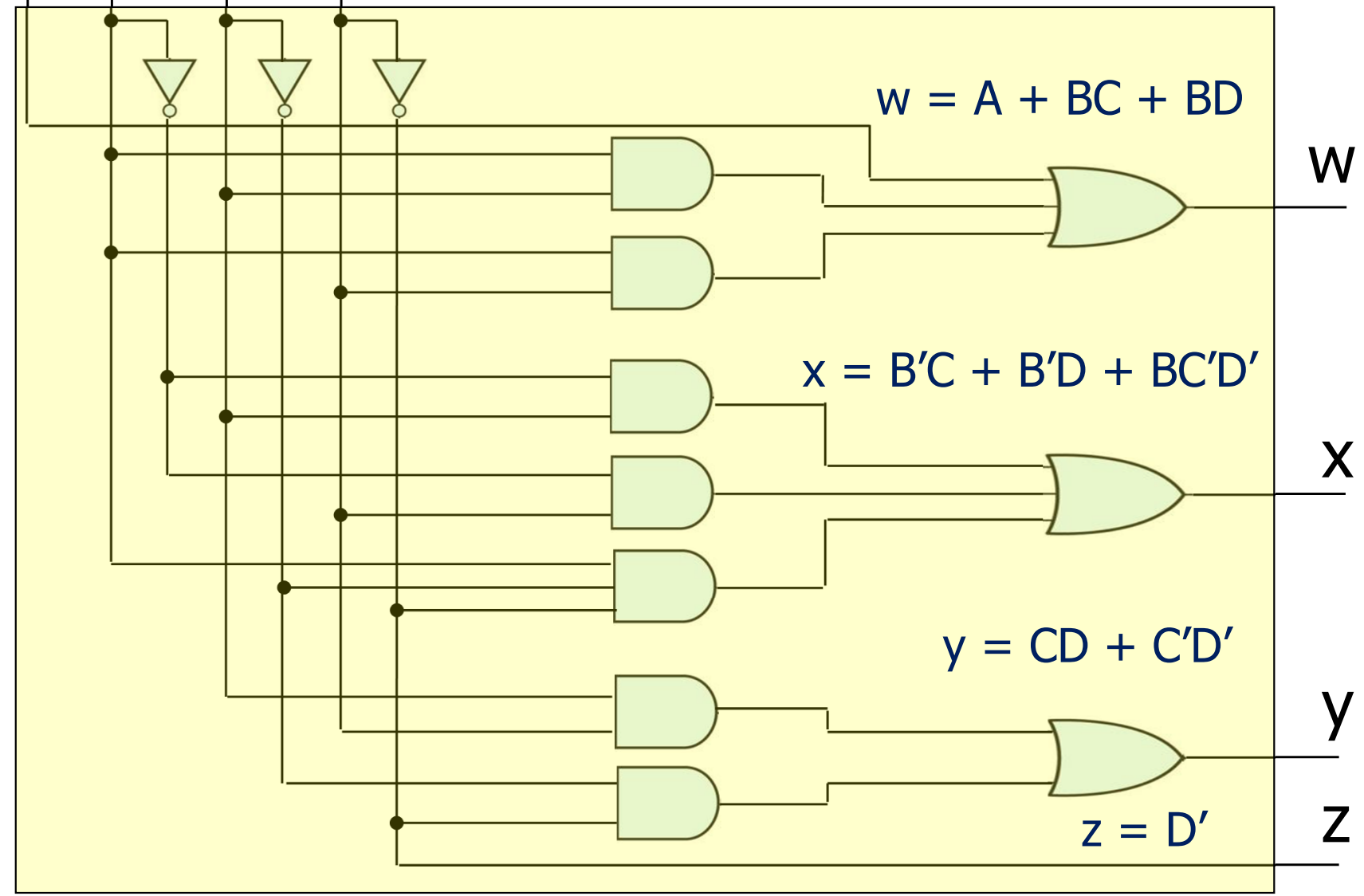
y = C'D'+ CD

AB\CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	x	x	x	x
10	1	0	x	x

z = D'

Example 2 (Cont.)

Step 4: Circuit Drawing





Binary Adders and Subtractors

Half Adder

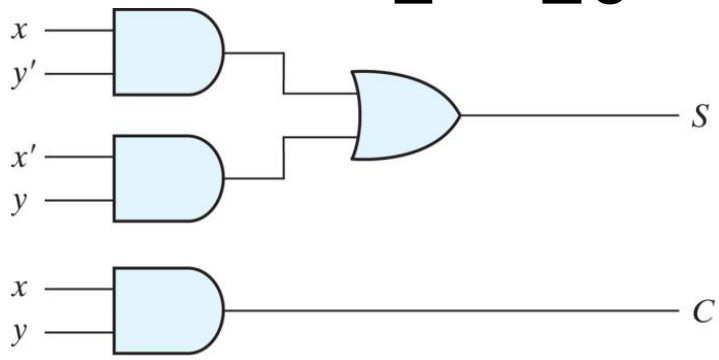
Add two binary numbers

- x , y → single bit inputs
- S → single bit sum
- C → carry out

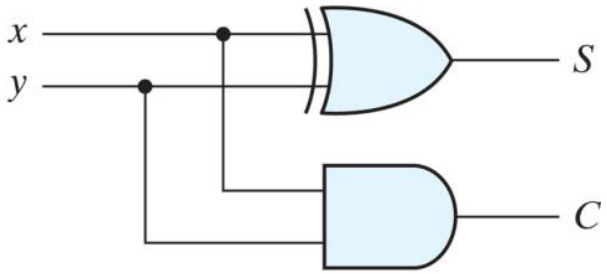
Dec	Binary
1	1
+1	+1
2	10

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



(a) $S = xy' + x'y$
$C = xy$



(b) $S = x \oplus y$
$C = xy$

Multiple-bit Addition

Consider adding a 4-bit number to a 4-bit number:

(single-bit adder are used for each bit position)

	A_3	A_2	A_1	A_0
A	0	1	0	1

	B_3	B_2	B_1	B_0
B	0	1	1	1

	1	1	1	
A	0	1	0	1
B	0	1	1	1
	1	1	0	0

C_{i+1} C_i

A_i

$+B_i$

S_i

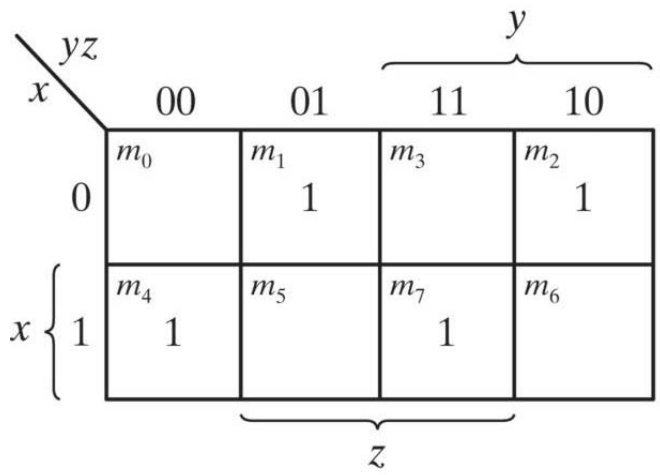
Each bit position creates a sum and carry

Full Adder

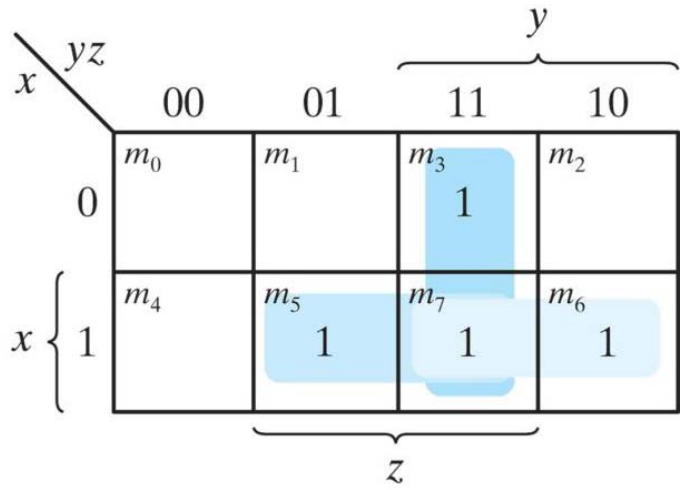
Full adder includes a carry–in z

Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

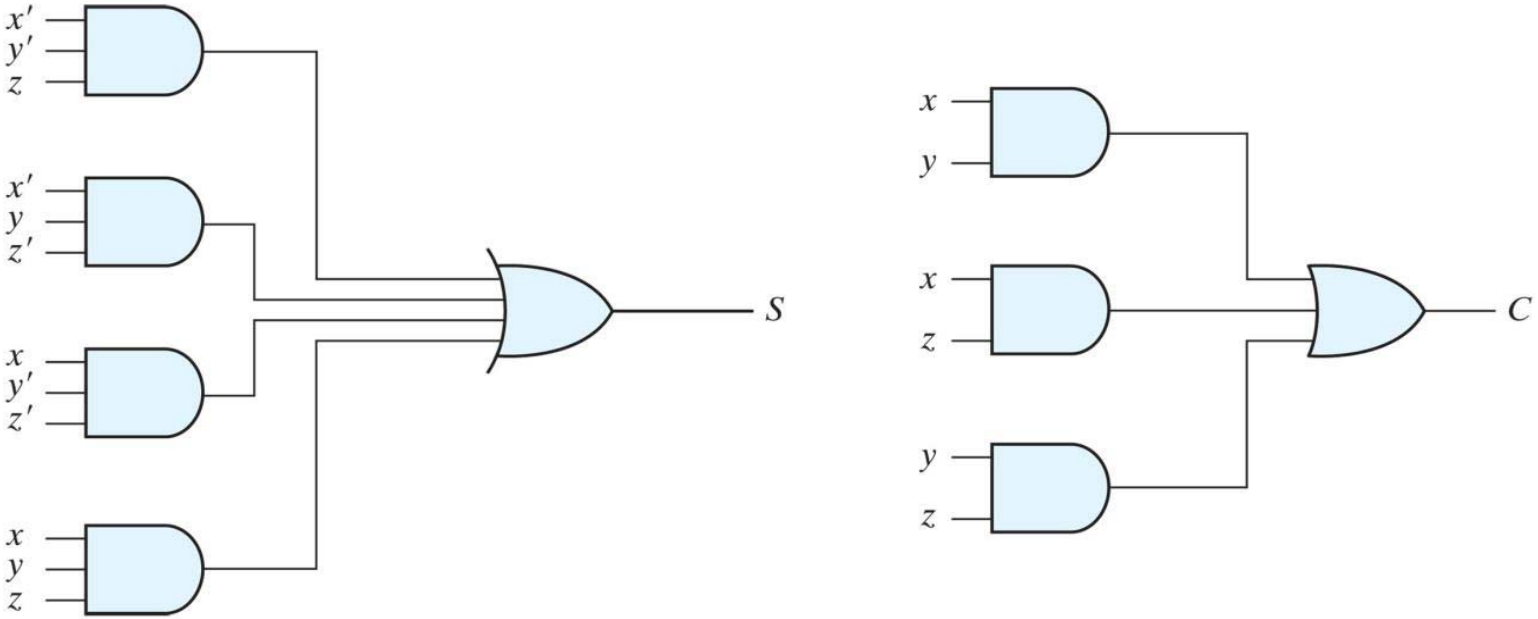


$$S = x'y'z + x'yz' + xy'z' + xyz$$



$$C = xy + xz + yz$$

Full Adder (cont.)



Full Adder (cont.)

The AND/OR representations of *S* and *C* can be reduced into XORs as follows:

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$= z\,(x'\,y' + x\,y) + z'\,(x'\,y + x\,y') \quad = z\,(x \odot y) + z'\,(x \oplus y)$$

$$= z\,(x \oplus y)' + z'\,(x \oplus y) \quad = z \oplus x \oplus y$$

$$C = xy + x'yz + xy'z$$

$$= xy + z\,(x'y + xy')$$

$$= xy + z\,(x \oplus y)$$

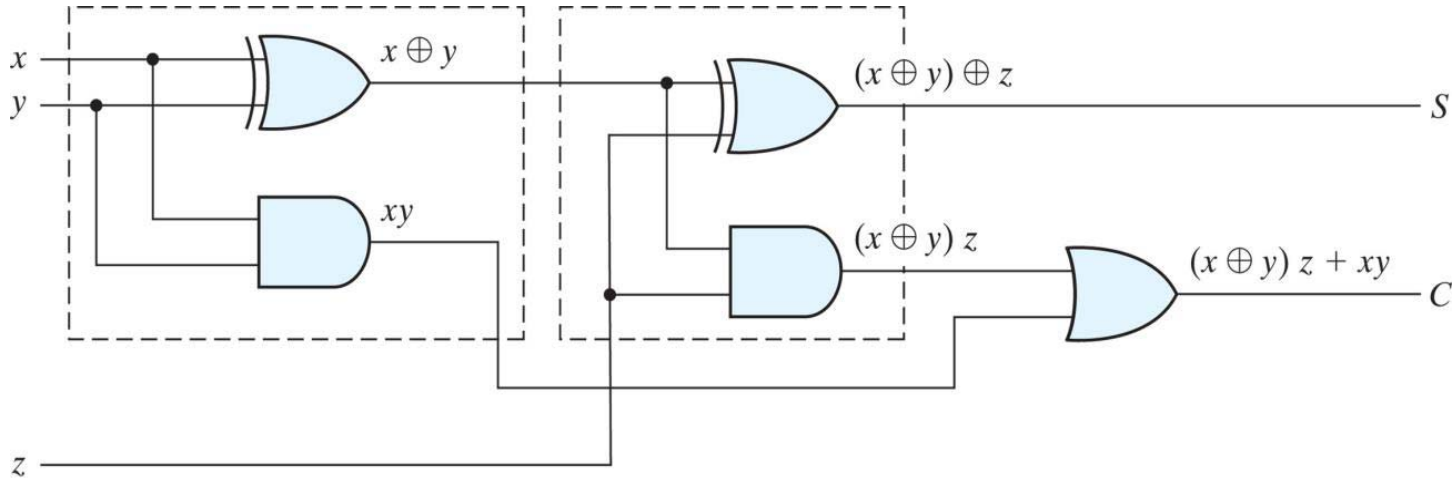
Therefore, the final equations are:

$$S = z \oplus x \oplus y$$

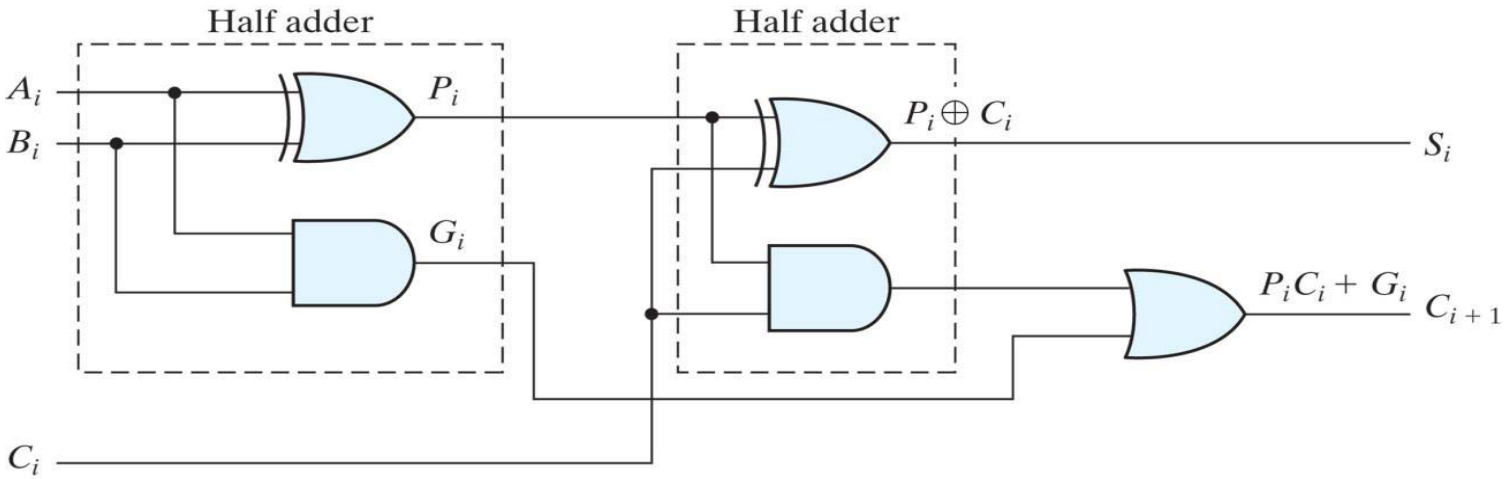
$$C = xy + z\,(x \oplus y)$$

Full Adder (cont.)

$$S = z \oplus x \oplus y \quad \textit{and} \quad C = xy + z\,(x \oplus y)$$



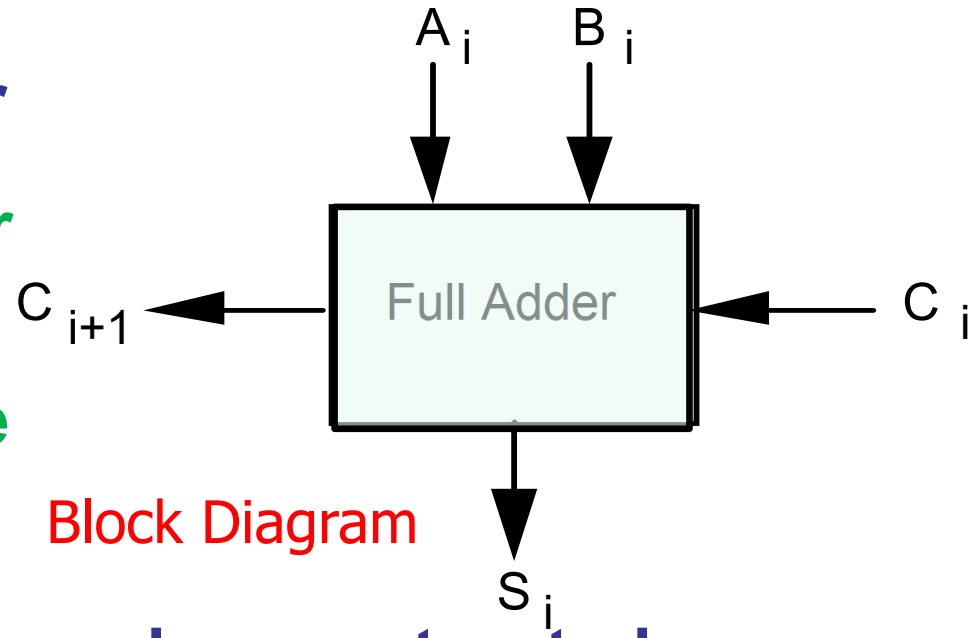
Therefore, the full adder can be constructed from two half adders and an OR gate.



Full Adder (cont.)

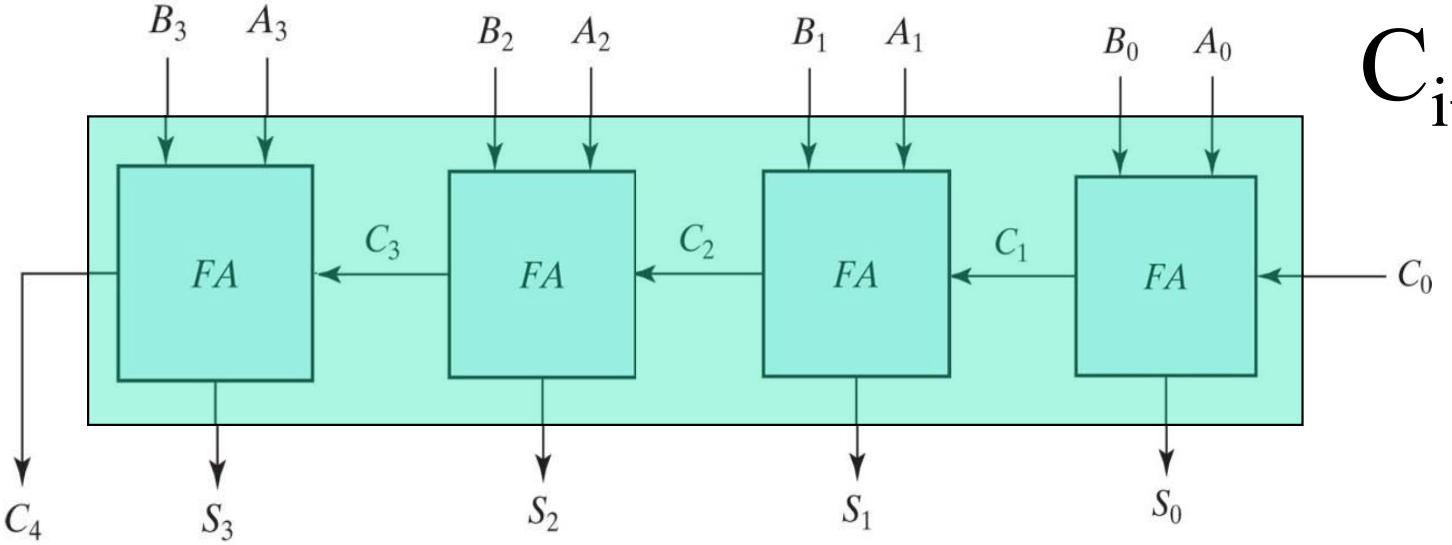
Putting it all together

- Single-bit full adder
- Common piece of computer hardware



Block Diagram

Accordingly, 4-bit Adder can be constructed as:

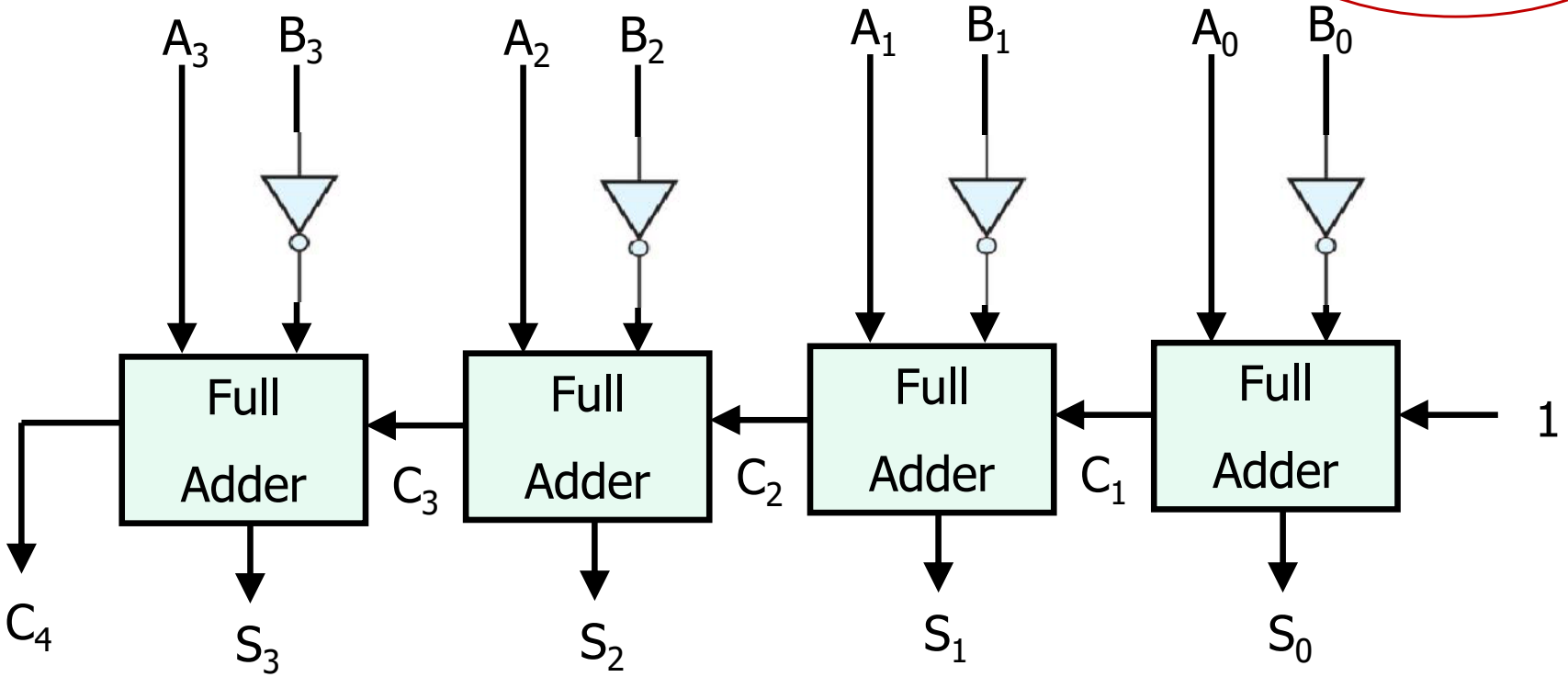


$$C_{i+1} \quad C_i \quad A_i \quad +B_i \quad S_i$$

Subtractor

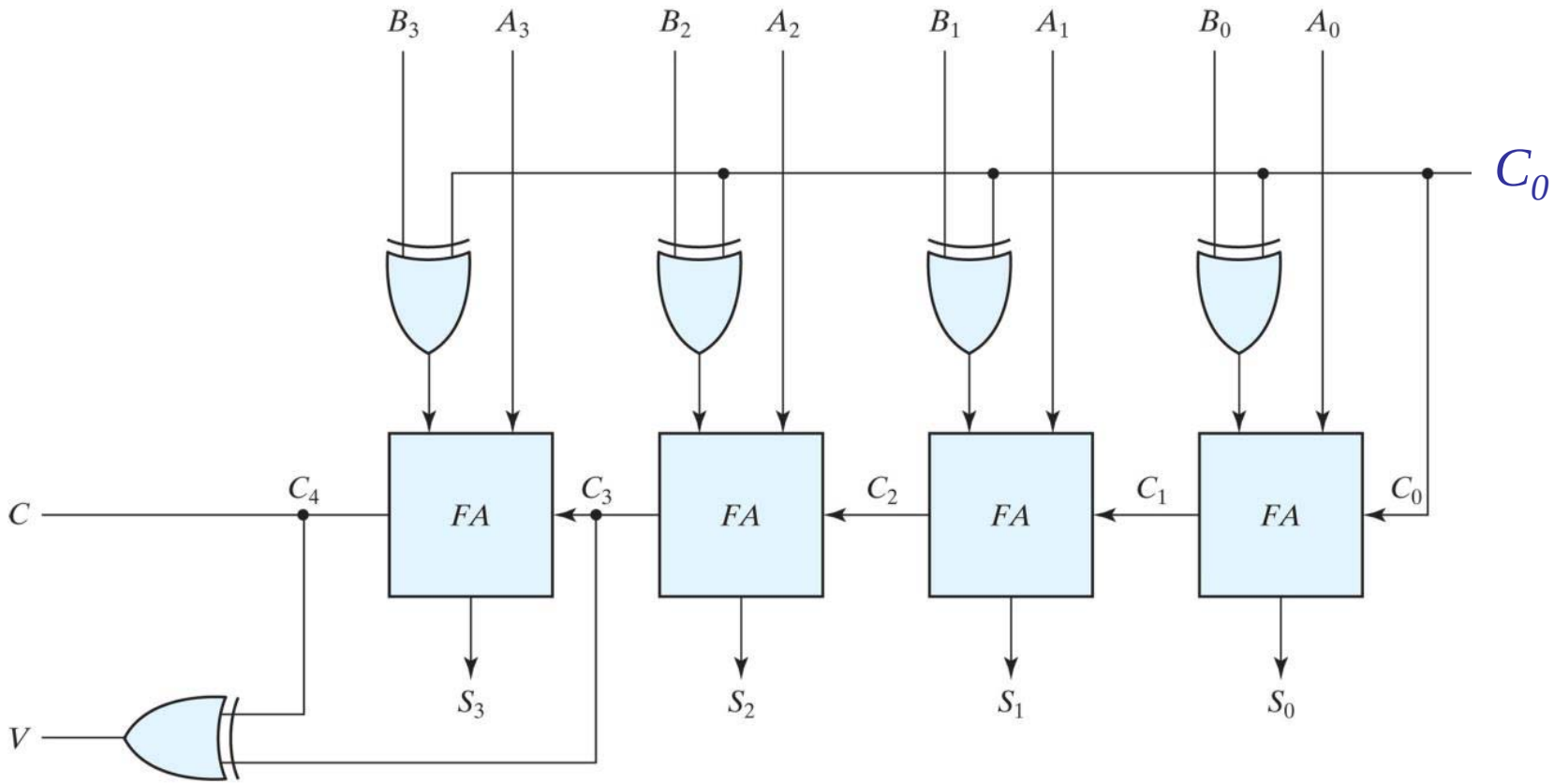
- **Subtracting a number is similar**
  - **Perform 2's complement**
  - **Perform addition**
- **Can we augment the adder with 2's complement hardware?**

$$
\begin{array}{rl}
 & A_3\ A_2\ A_1\ A_0 \\
- & B_3\ B_2\ B_1\ B_0 \\
\hline
\text{2's complement} & A_3\ A_2\ A_1\ A_0 \\
+ & B'_3\ B'_2\ B'_1\ B'_0 \\
 & +\qquad\quad 1 \\
\hline
\end{array}
$$



Adder-Subtractor

- **4-bit Adder-subtractor (with overflow detection).**
  - **When $C_0 = 0$, the circuit is used as an Adder.**
  - **When $C_0 = 1$, the circuit is used as a Subtractor.**



Overflow in 2's Complement Addition

When two values of the same sign are added:

- Result won't fit in the number of bits provided
- Result has the opposite sign

00	01	11	10	00	11
0010	0011	1110	1101	0010	1110
0011	0110	1101	1010	1100	0100
---	---	---	---	---	---
0101	1001	1011	0111	1110	0010
+2	+3	-2	-3	2	-2
+3	+6	-3	-6	-4	+4
5	-7	-5	7	-2	2
	OFL		OFL		

Summary

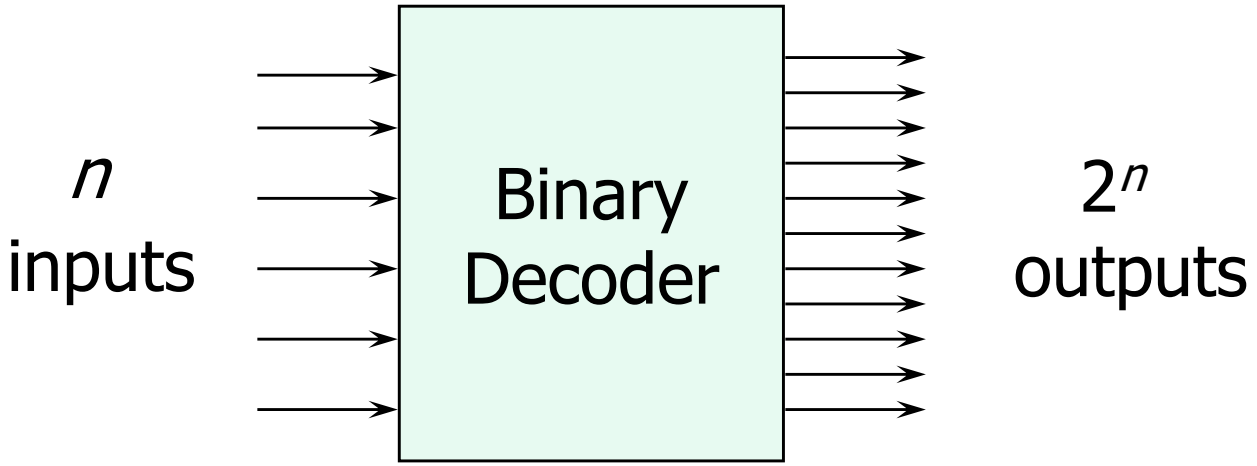
- Addition and subtraction are fundamental to computer systems
- Create a single bit adder/subtractor
  - Chain the single-bit hardware together to create bigger designs
- The approach is called ripple-carry addition
  - Can be slow for large designs
- Overflow is an important issue for computers
  - Processors often have hardware to detect overflow



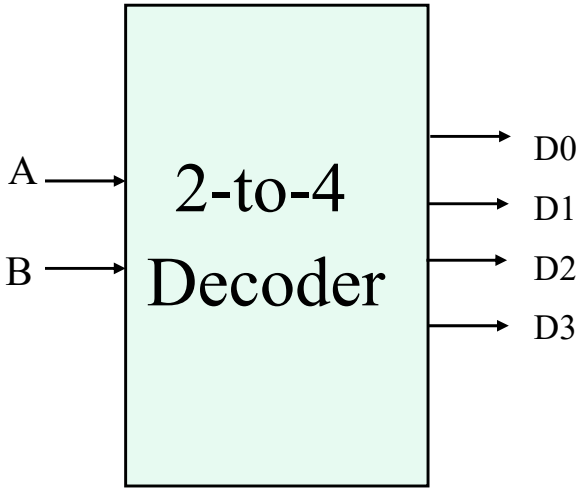
Binary Decoder and Encoder

Binary Decoder

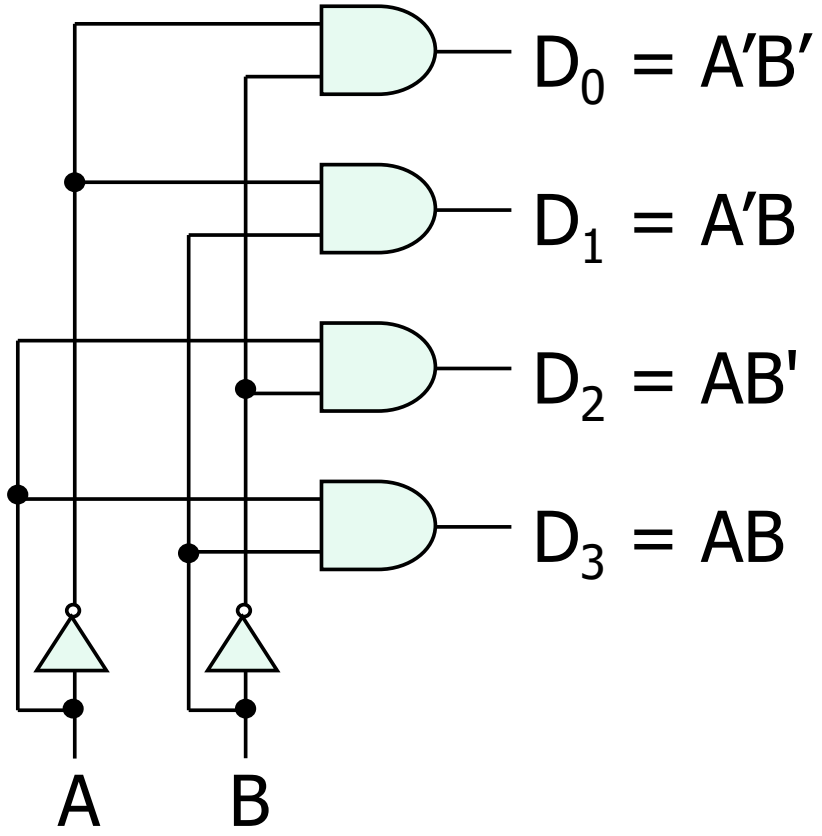
- Black box with *n* input lines and 2^n output lines.
- Only one output is a 1 for any given input.



2-to-4 Binary Decoder

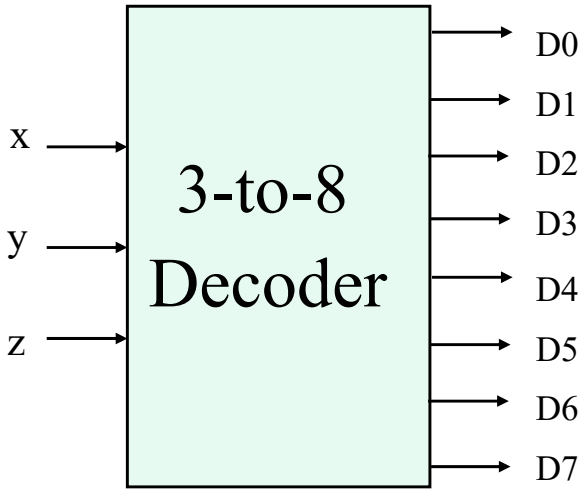


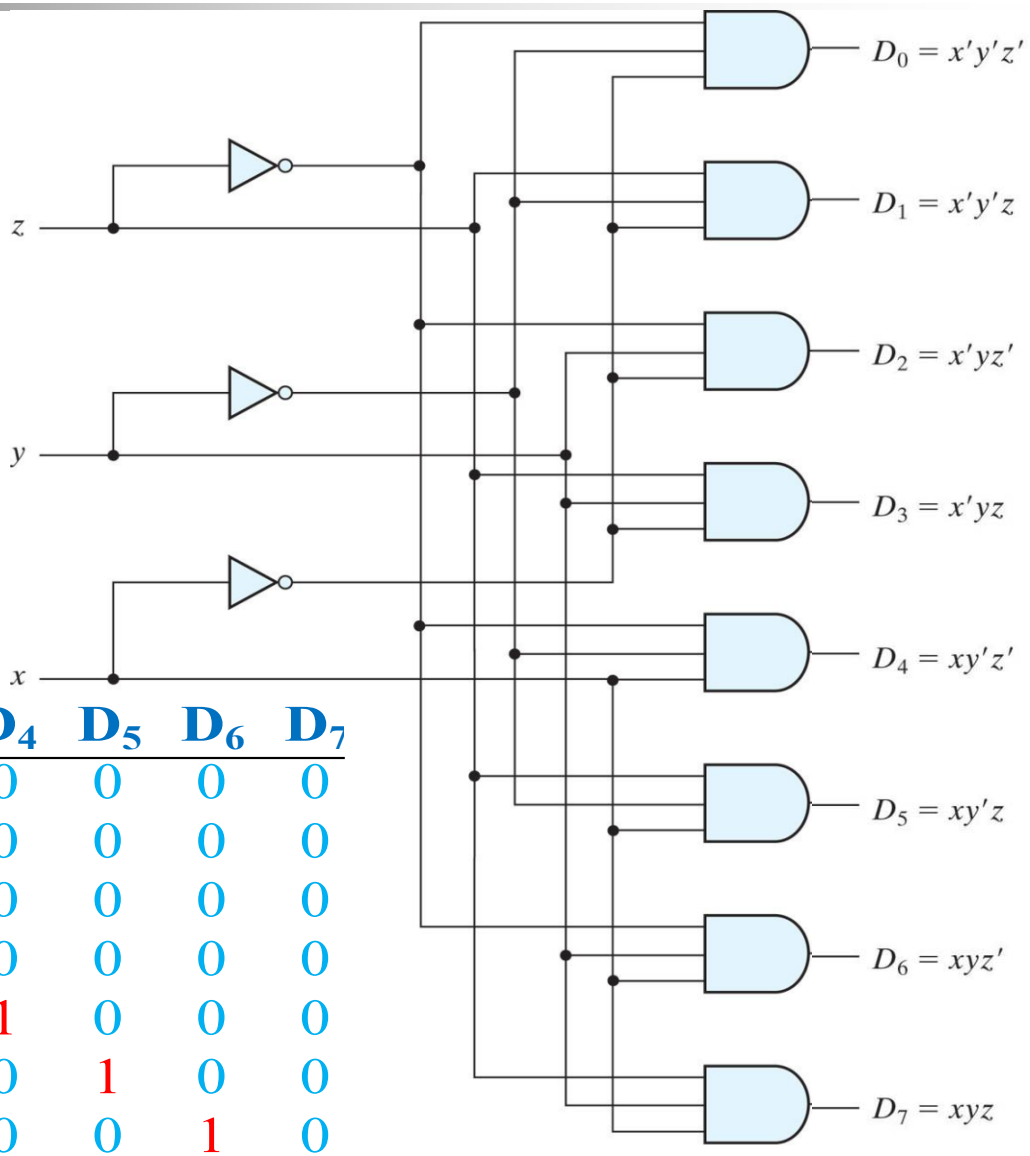
A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



Note: Each output is a 2-variable minterm (A'B', A'B, AB' or AB)

3-to-8 Binary Decoder



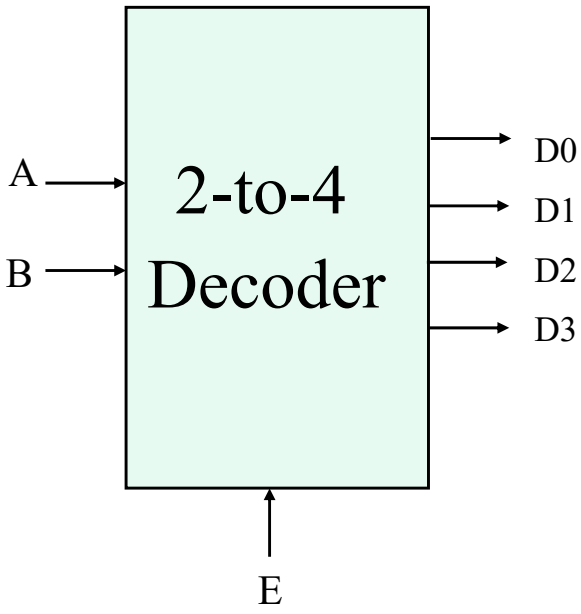


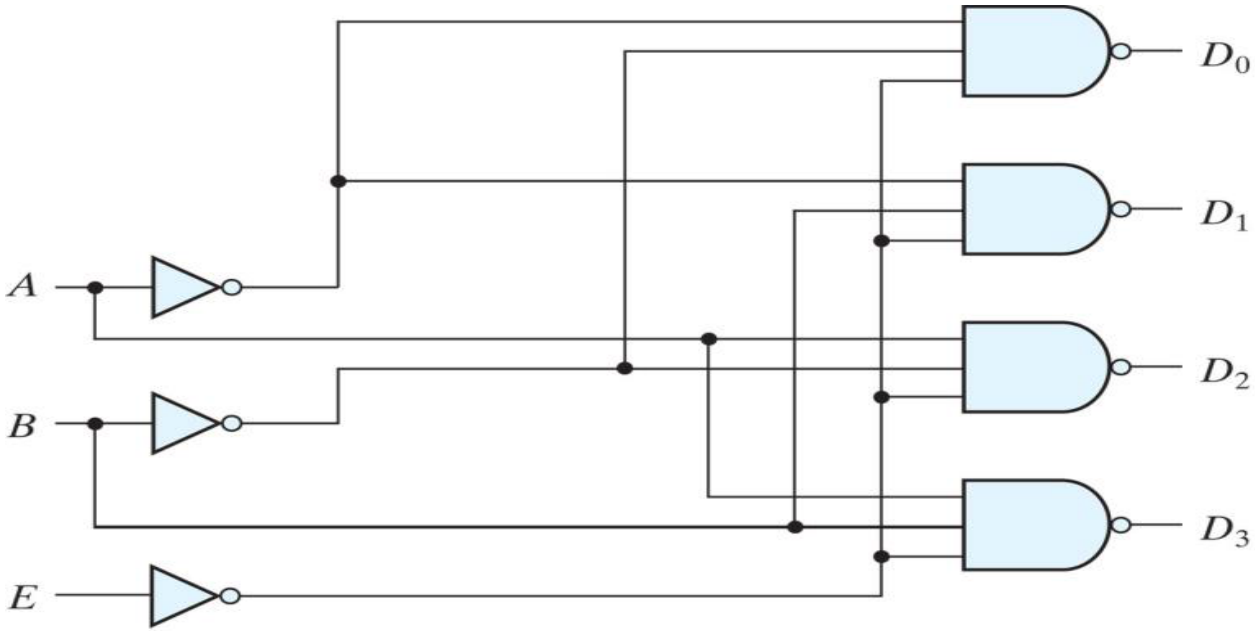
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Building a binary decoder with NAND gates

- 2-to-4 decoder with enable
- Add enable signal (E)
- Note: using NAND Gates makes only one 0 be active if the enable E = 0.

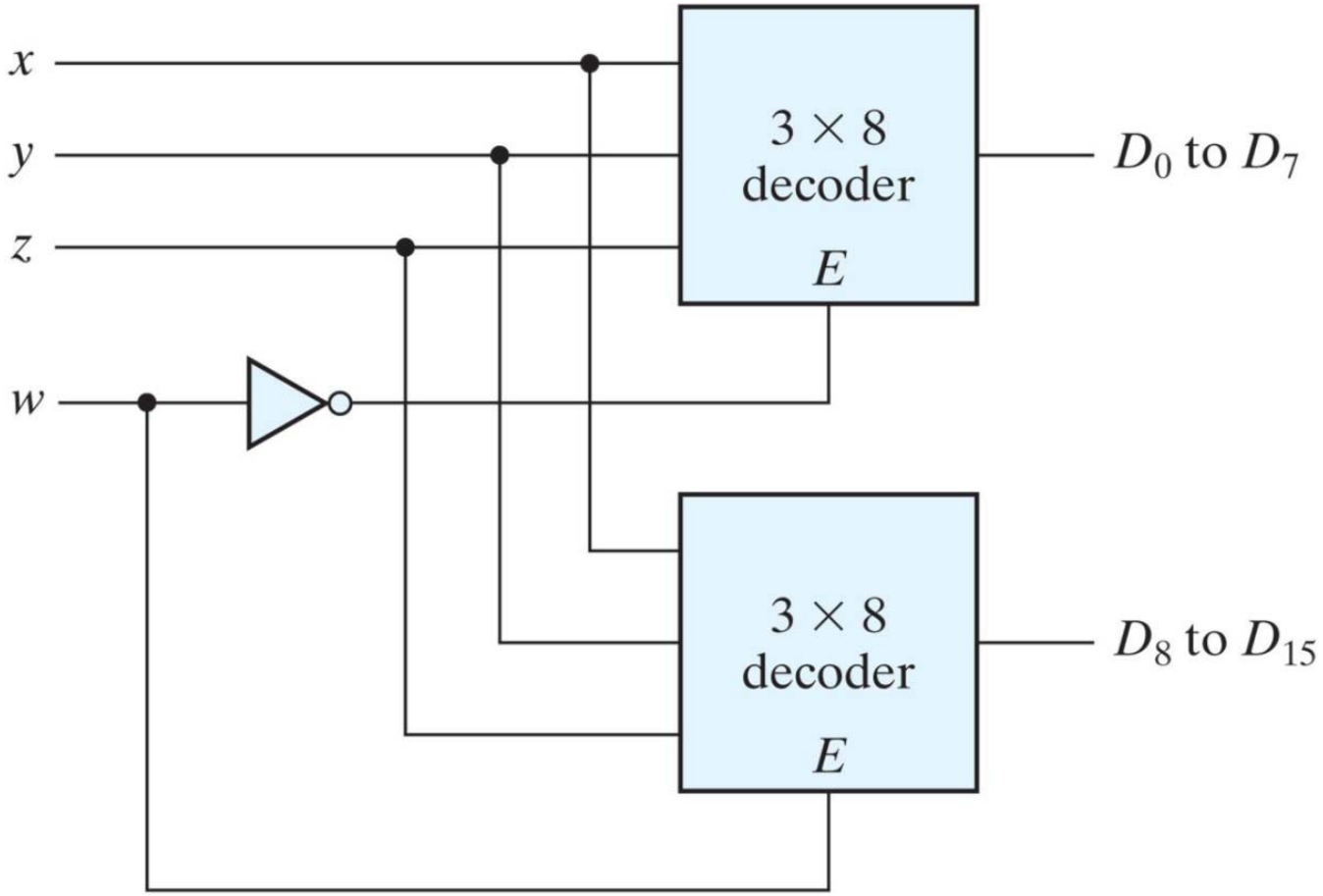
E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1





Use two 3-to-8 decoders to make 4-to-16 decoder

- The enable is used as the 4th line.
- In this case, only one decoder can be active at a time.



Implementing Functions Using Decoders

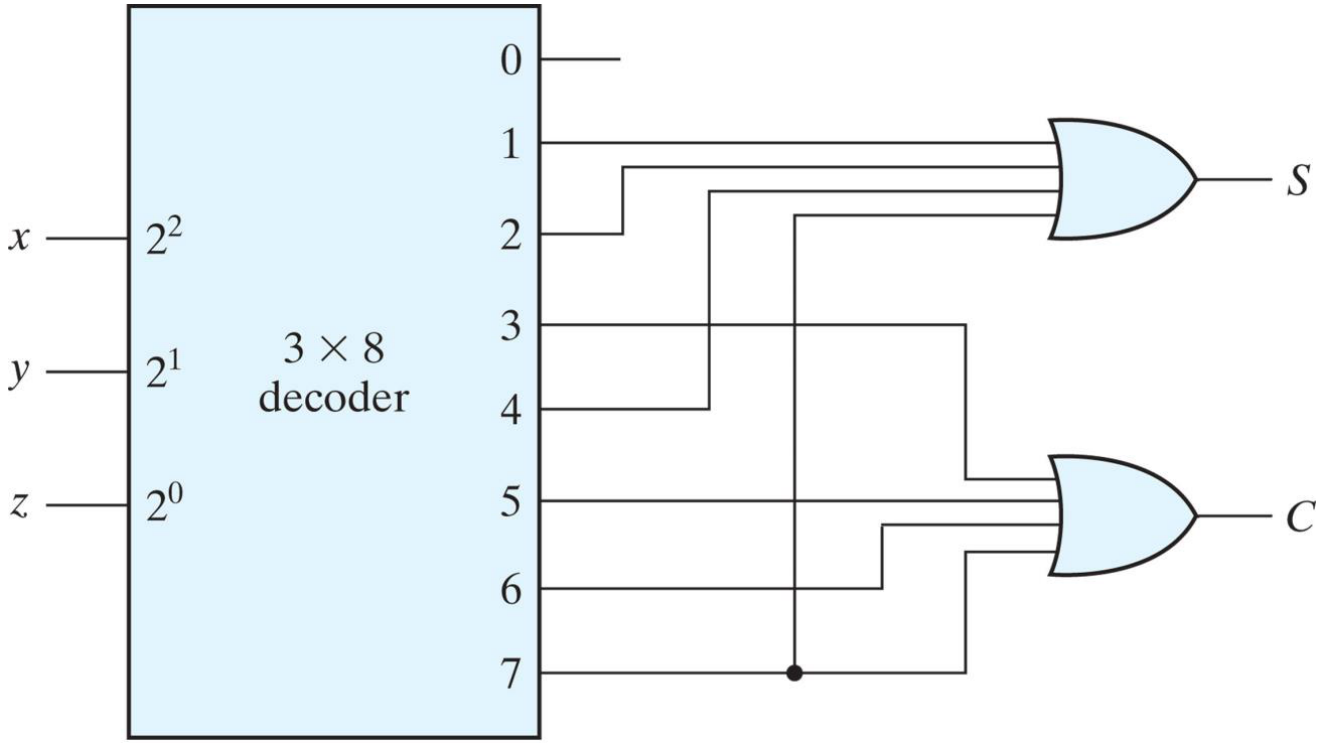
- Any n-variable logic function can be implemented using a single *n-to-*2^n decoder to generate the minterms.
  - OR gate forms the sum
  - The output lines of the decoder corresponding to the minterms of the function are used as inputs to the OR gate
- Any combinational circuit with n inputs and m outputs can be implemented with an *n-to-*2^n decoder with *m* OR gates.
- Suitable when a circuit has many outputs, and each output function is expressed with few minterms.

Example

Implement the function of the Full adder using a decoder.

Sum: S(x, y, z) = Σ (1,2,4,7) Carry: C(x, y, z) = Σ (3,5,6,7)

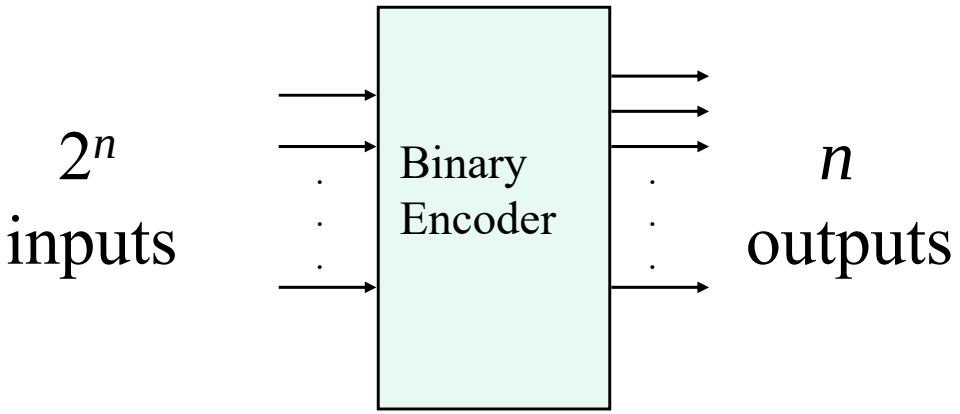
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



Note: if you have the function, you do not need to construct the truth table.

Binary Encoder

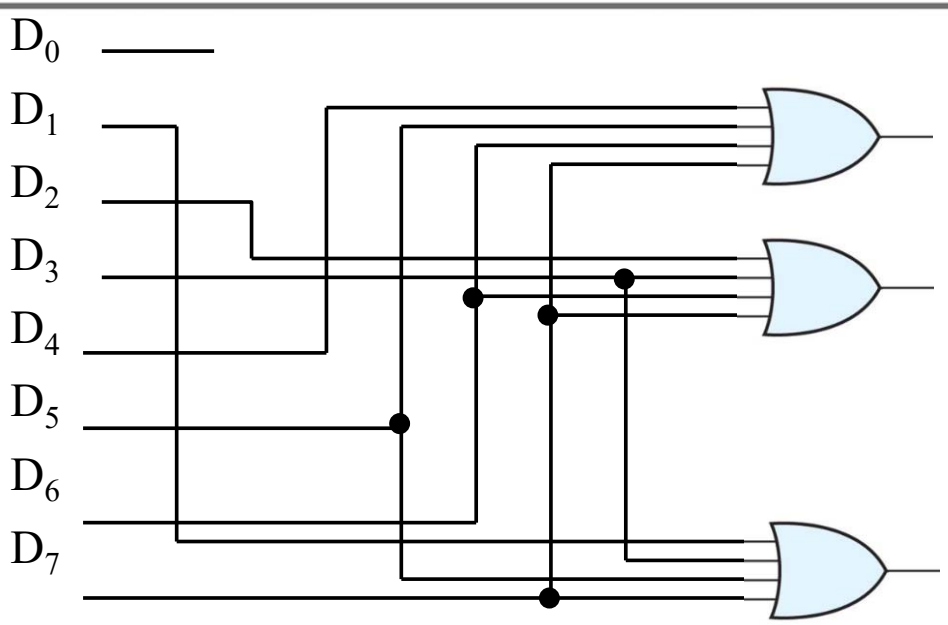
- If a decoder's output code has fewer bits than the input code, the device is usually called an encoder.
  - e.g. 2^n-to-n
- The simplest encoder is a 2^n-to-n binary encoder
  - One of 2^n inputs = 1
  - Output is an n-bit binary number



8-to-3 Encoder

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1



$x = D_4 + D_5 + D_6 + D_7$

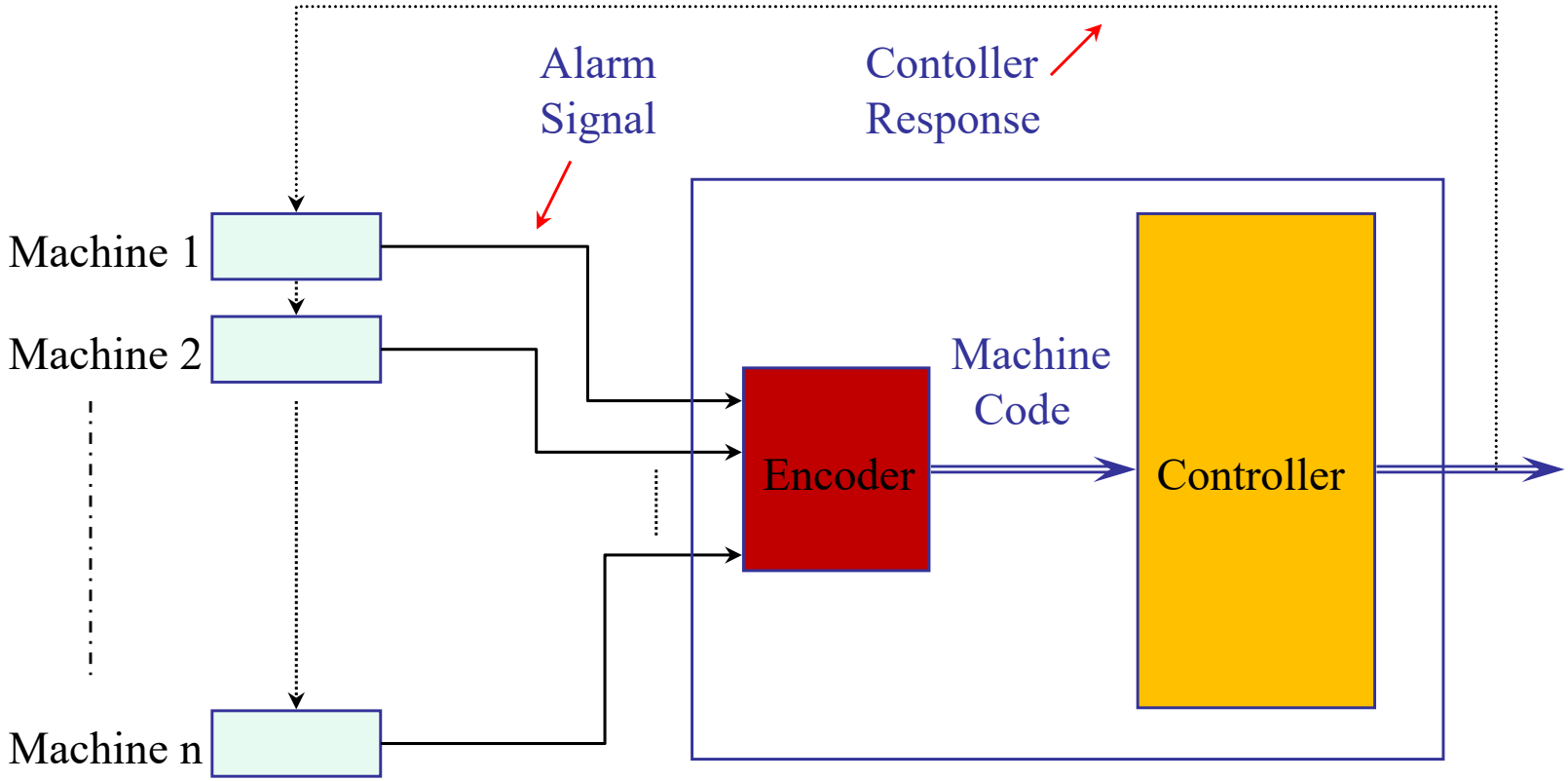
$y = D_2 + D_3 + D_6 + D_7$

Note: At any one time, only one input line has a value of 1.

$z = D_1 + D_3 + D_5 + D_7$

Encoder Application (Monitoring Unit)

- Encoder identifies the requester and encodes the value.
- Controller accepts digital inputs.



Summary

- Decoders allow for generation of a single binary output from an input binary code.
- For an n-input binary decoder there are 2^n outputs.
- Decoders are widely used with memory.
- Encoders can be used for data compression.



Binary Magnitude Comparators

Magnitude Comparator

- Compares two numbers: *A* and *B*
  - Result: $A > B$, $A = B$, or $A < B$
- Truth table entries for *n*-bit numbers = 2^{2n} entries
  - Impractical for design
- How can we determine that two numbers are equal?
  - $A_3\,A_2\,A_1\,A_0$ and $B_3\,B_2\,B_1\,B_0$ are equal iff
  - $A_3 = B_3$ and $A_2 = B_2$ and $A_1 = B_1$ and $A_0 = B_0$
- New function: x_i indicates if $A_i = B_i$
  - $x_i = A_iB_i + A'_iB'_i$ (X-NOR)
  - Thus, $(A = B) = x_3\,x_2\,x_1\,x_0$
  - What about $A < B$ and $A > B$?

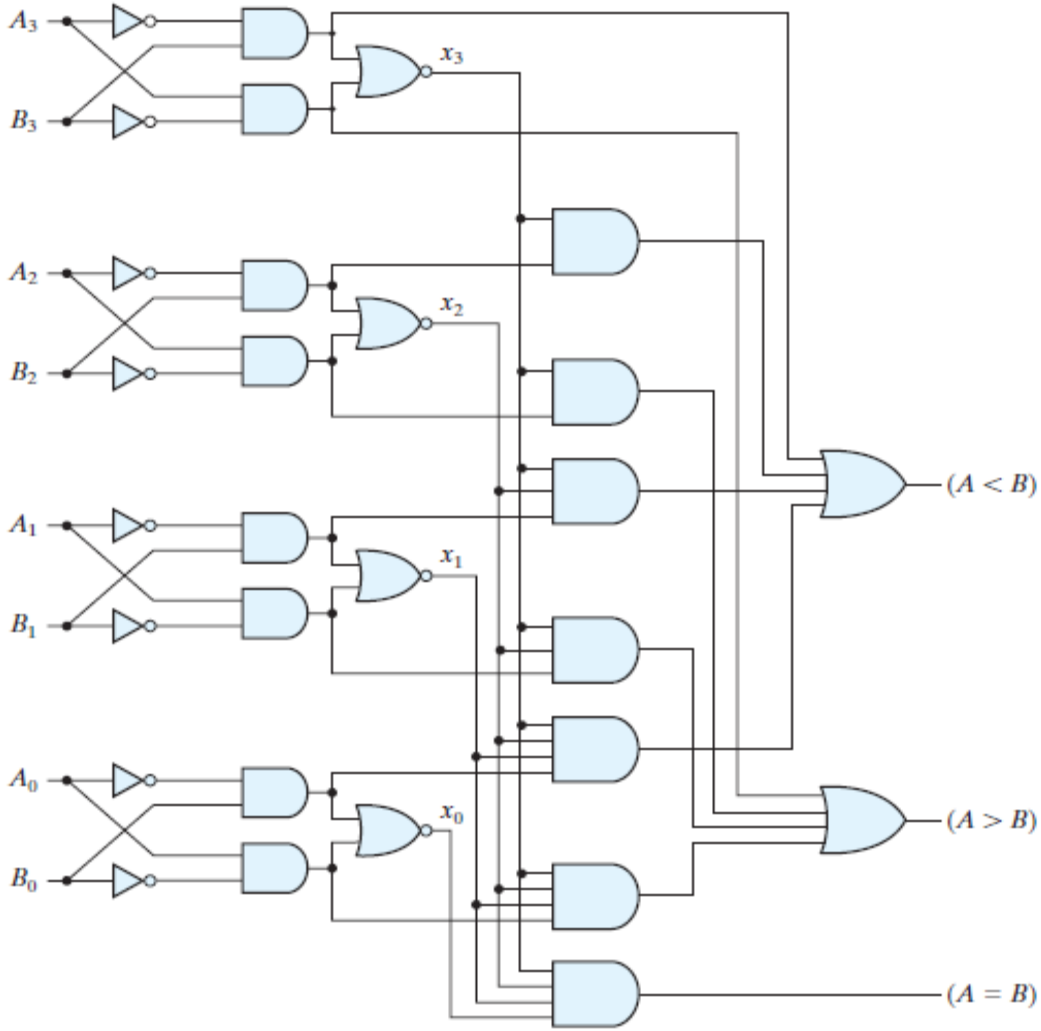
Magnitude Comparator (cont.)

- A > B
  - How can we tell that A > B?
  - Look at MSB where A and B differ
    - ✓ If A = 1 and B = 0, then A > B
    - ✓ If not, then $A \leq B$
  - Assume $n = 4$
  - $(A>B) = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$
- A < B
  - The same as A > B but A and B are exchanged.
  - $(A<B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B_1 + x_3 x_2 x_1 A'_0 B_0$

Note: The comparasion must be started from MSB.

Magnitude Comparator (cont.)

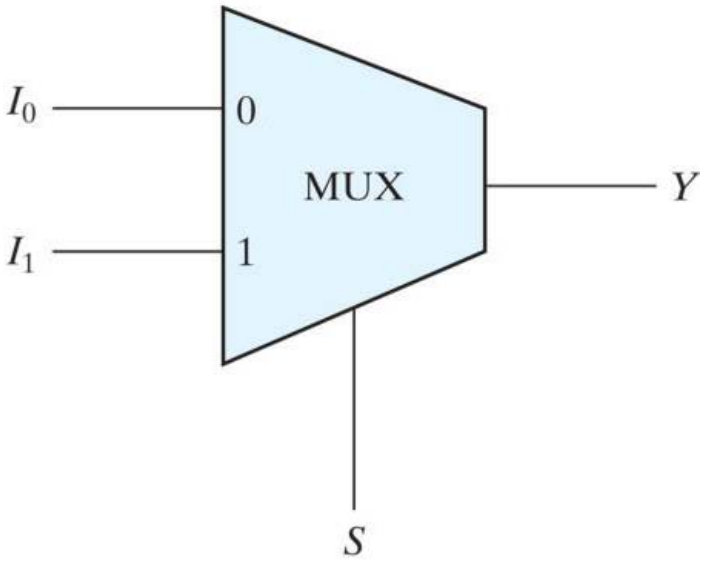
- $(A>B) = A_3 B'_3 + x_3 A_2 B'_2 + x_3 x_2 A_1 B'_1 + x_3 x_2 x_1 A_0 B'_0$
- $(A<B) = A'_3 B_3 + x_3 A'_2 B_2 + x_3 x_2 A'_1 B_1 + x_3 x_2 x_1 A'_0 B_0$
- $(A = B) = x_3 x_2 x_1 x_0$

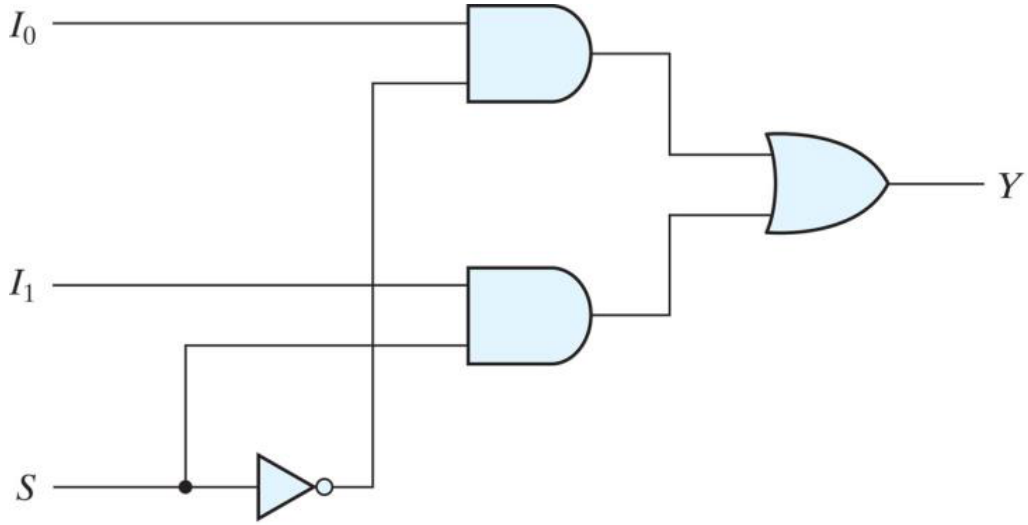


Binary Multiplexers

Binary Multiplexers

- Select an input value with one or more select bits
- Use for transmitting data
- Allow for conditional transfer of data
- It is acronymed as MUX
- Example: 2-to-1 MUX

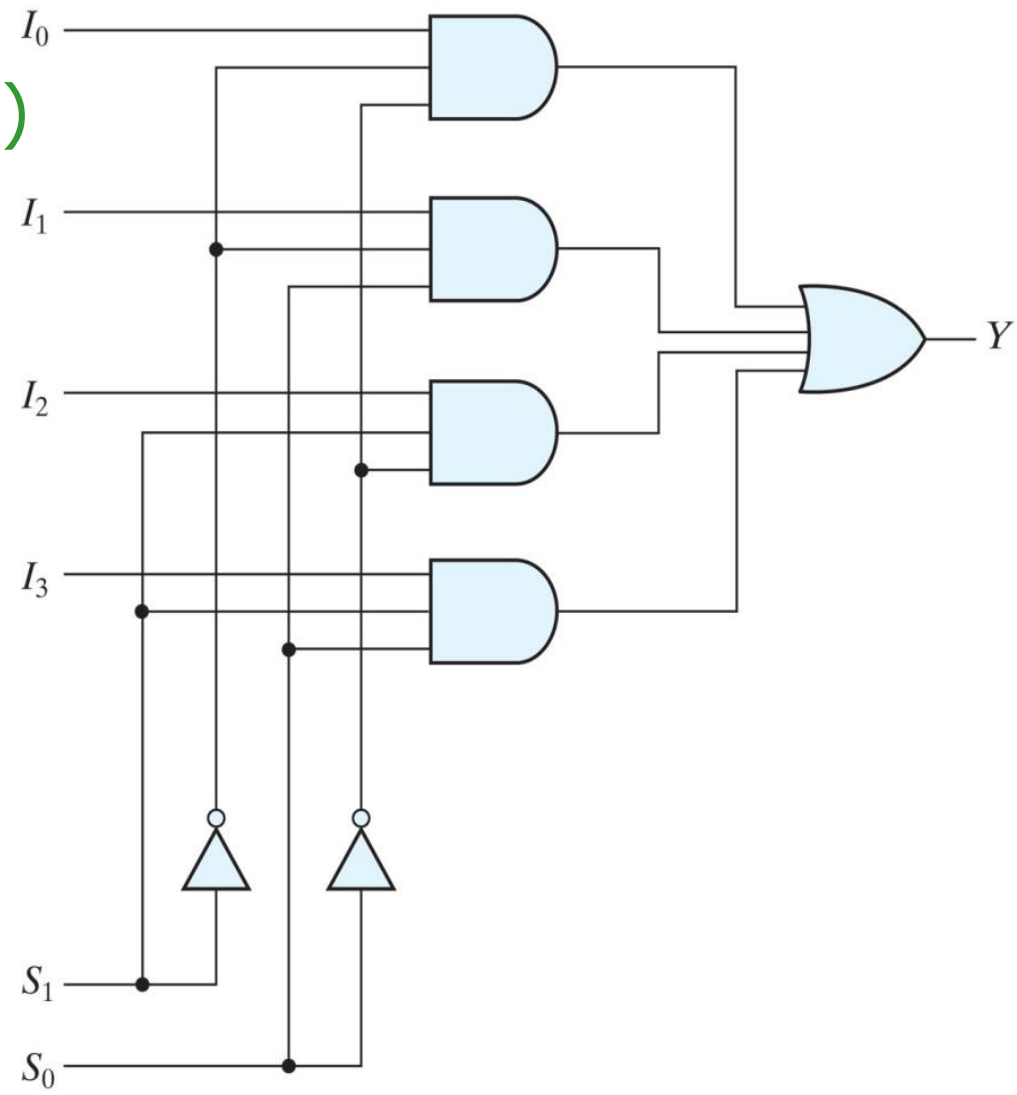




Multiplexers (cont.)

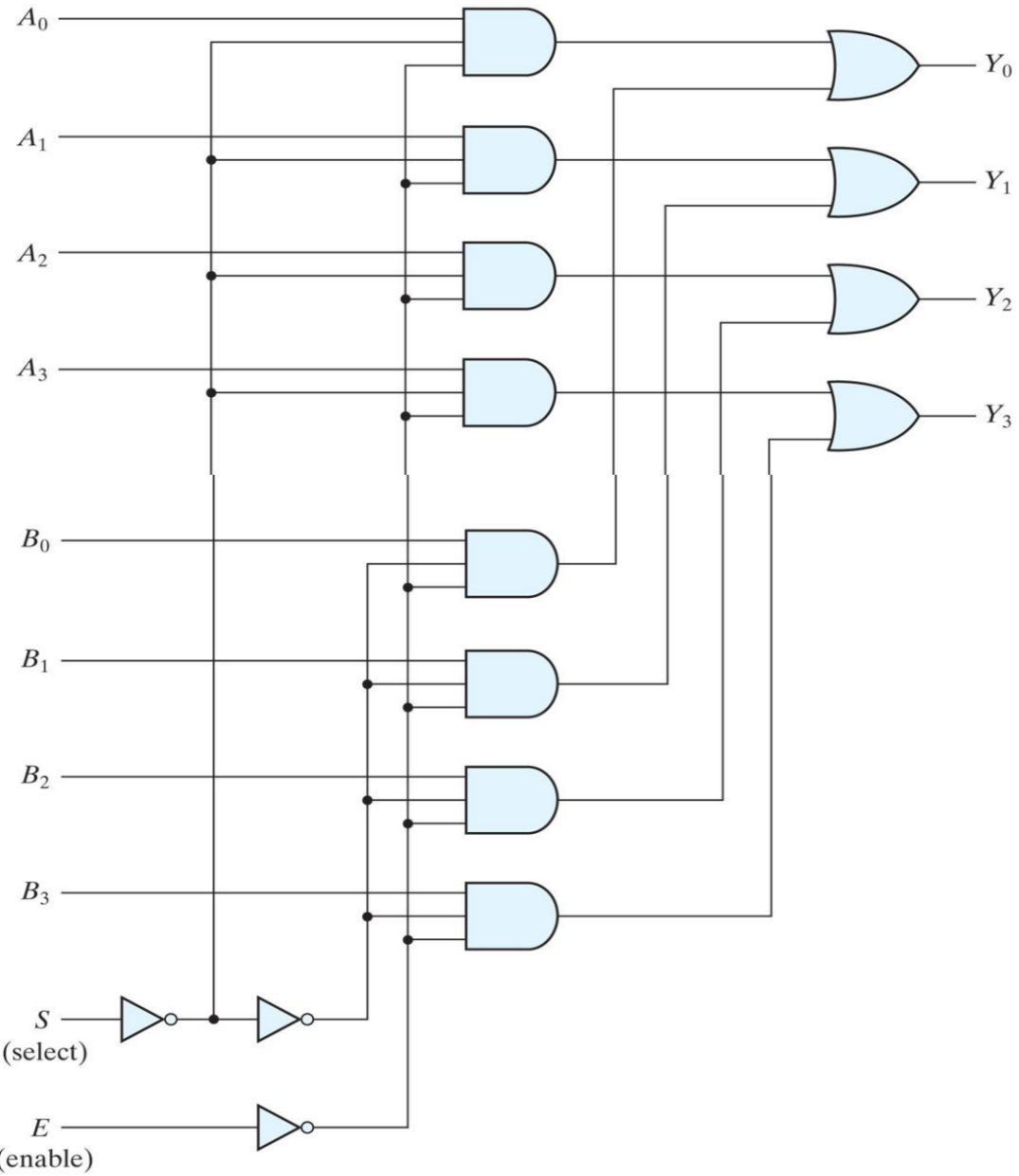
- 4-to-1 Multiplexer
  - 4 input lines (I_0, I_1, I_2, I_3)
  - 2 selection lines (S_0, S_1)
  - One output line (Y)
  - Function table

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



Multiplexers (cont.)

- Quadruple 2-to-1 MUX
  - 4-bit inputs
  - 4-bit outputs
  - one selection bit (*S*)
  - Enable bit (*E*)

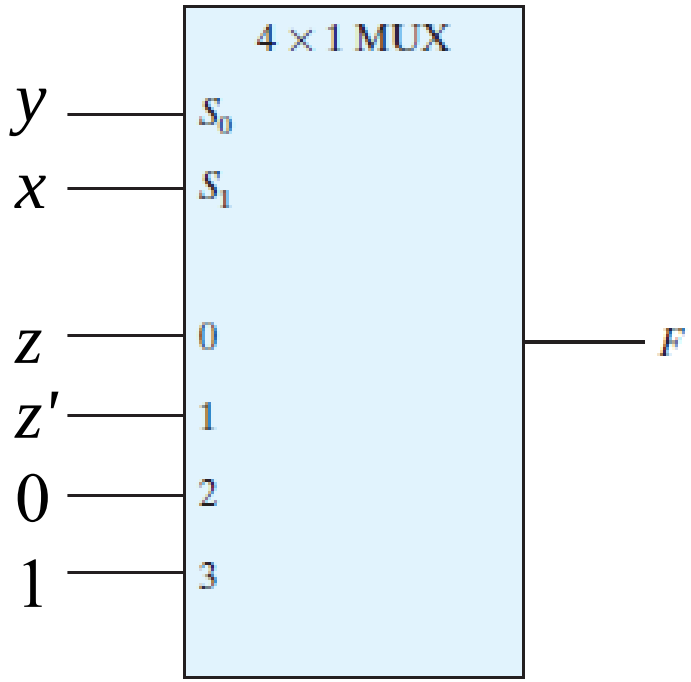


E	*S*	Output *Y*
1	X	all 0's
0	0	select *A*
0	1	select *B*

Function table

Implementing Boolean Functions with MUXs

- Example: Implement the Boolean function $F(x, y, z) = \Sigma(1, 2, 6, 7)$ using 4 x 1 MUX.



S_1 S_0

x	y	z	F	
0	0	0	0	$I_0 = z$
0	0	1	1	
0	1	0	1	$I_1 = z'$
0	1	1	0	
1	0	0	0	$I_2 = 0$
1	0	1	0	
1	1	0	1	$I_3 = 1$
1	1	1	1	

- Set "data" input lines of multiplexer to function values
- Connect input variable to "select" inputs
- Set the inputs of multiplexer.

Implementing a 4-Input Function with a MUX

- Example: Implement the Boolean function F(A, B, C, D) = Σ(1, 3, 4, 11, 12, 13, 14, 15) using 8 x 1 MUX.

S_2	S_1	S_0		
A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

$I_0 = D$

$I_1 = D$

$I_2 = D'$

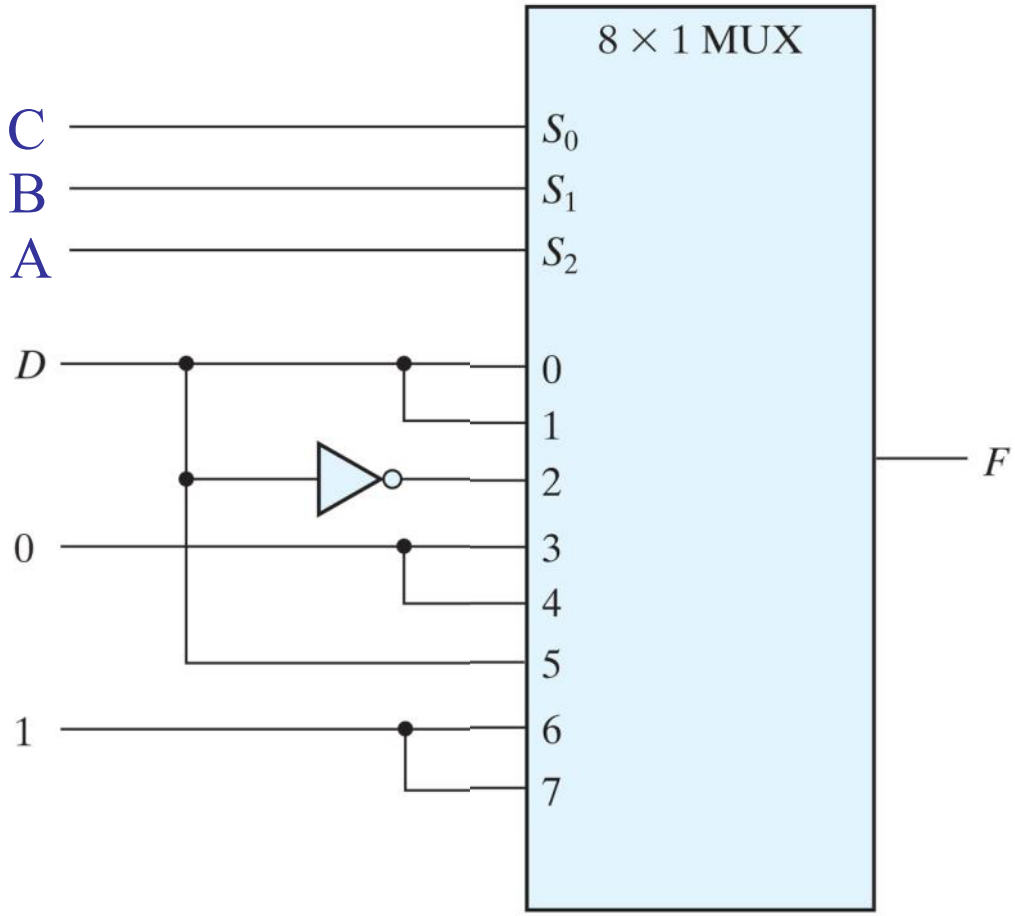
$I_3 = 0$

$I_4 = 0$

$I_5 = D$

$I_6 = 1$

$I_7 = 1$

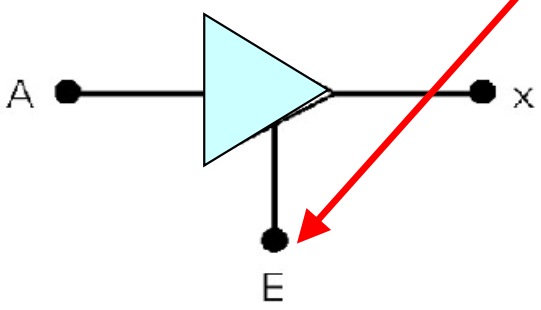




Three-state Gate

Three-state Gate

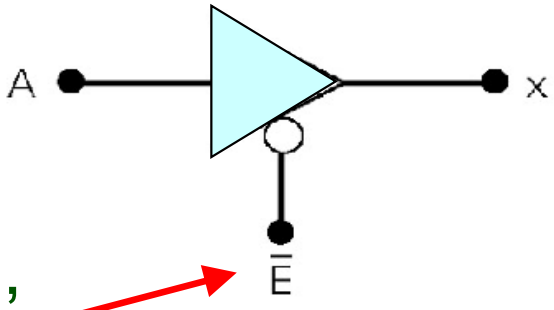
- Output: 0, 1, and high-impedance (open circuit)
- If the select input (E) is 0, the three-state gate has no output



E	x
0	Hi-Z
1	A

Opposite true here,

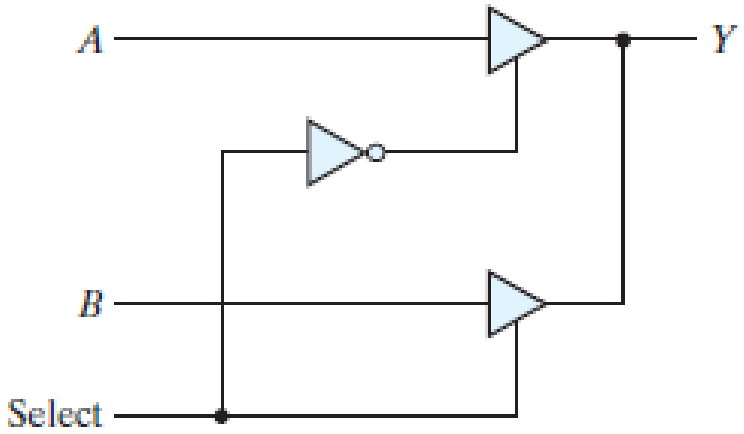
No output if E is 1



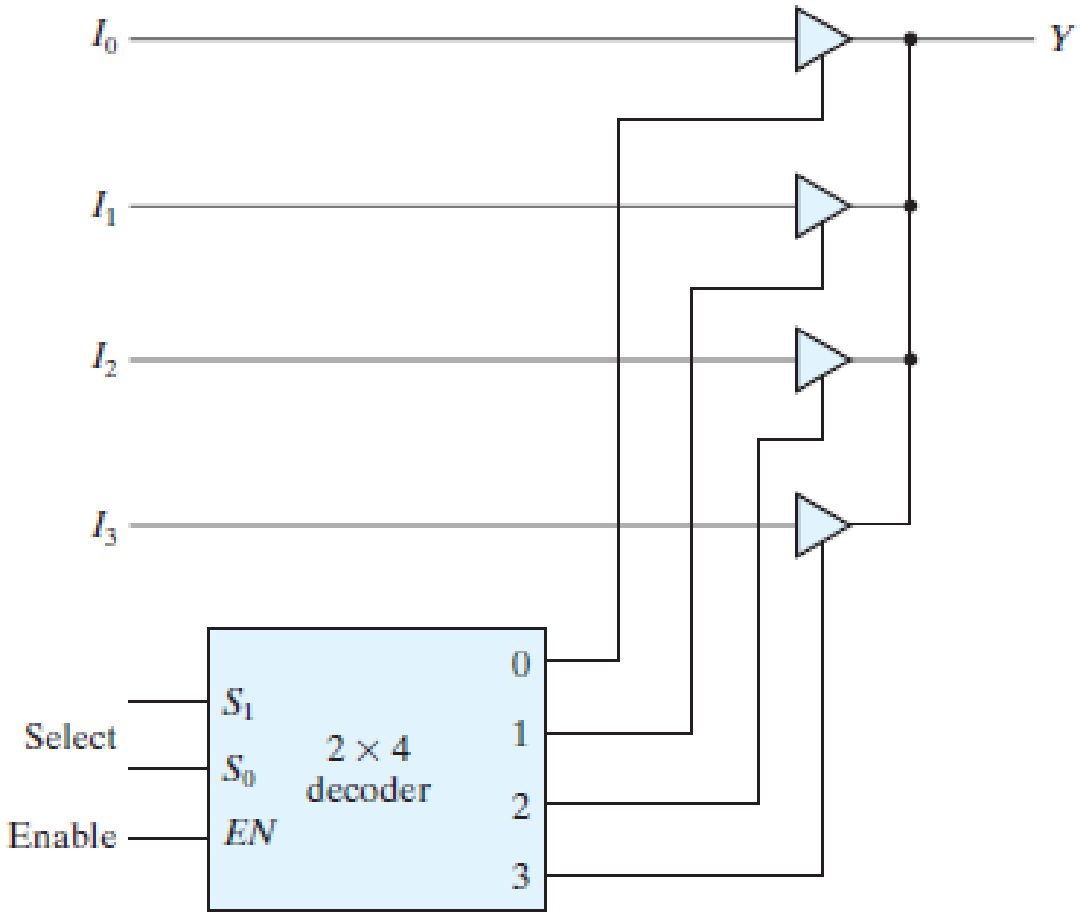
$\bar{E}$	x
0	A
1	Hi-Z

Three-state Gate (cont.)

- Multiplexers can be constructed with three-state gates.
- Two examples



(a) 2-to-1-line mux



(b) 4-to-1-line mux



Summary

- Magnitude comparators allow for data comparison
  - Can be built using AND-OR gates
  - Greater / Less than requires more hardware than equality
- Multiplexers are fundamental digital components
  - Can be used for implementing logic functions
  - Useful for datapaths
  - Scalable
- Tristate buffers have three types of outputs
  - 0, 1, high-impedence (Z)
  - Useful for datapaths